

# v31 Regenerated Calldata Verification



**September 25, 2026**

*This document has been prepared solely for the named client and it may not be relied upon by any third parties. The content in this document is provided for informational purposes only and does not constitute professional advice of any kind. Security assessments are limited by factors such as scope, time, and techniques used and therefore this document does not constitute a certification, guarantee, or warranty regarding the proper functioning of any system or the absence of security vulnerabilities.*

Copyright © 2026 Zeppelin Group Ltd.

# Table of Contents

|                                                             |    |
|-------------------------------------------------------------|----|
| Table of Contents                                           | 2  |
| Summary                                                     | 3  |
| Scope                                                       | 4  |
| Changes from the original calldata                          | 5  |
| The critical issue: ProtocolUpgradeHandler reinitialization | 6  |
| Status of previously reported issues                        | 7  |
| Fork verification                                           | 8  |
| Conclusion                                                  | 9  |
| Appendix                                                    | 10 |
| Issue Classification                                        | 10 |

# Summary

|           |                                  |                                |                |
|-----------|----------------------------------|--------------------------------|----------------|
| Type      | Layer 2 & Rollups                | Total Issues                   | 0 (0 resolved) |
| Timeline  | From 2026-09-22<br>To 2026-09-22 | Critical Severity Issues       | 0 (0 resolved) |
| Languages | Solidity                         | High Severity Issues           | 0 (0 resolved) |
|           |                                  | Medium Severity Issues         | 0 (0 resolved) |
|           |                                  | Low Severity Issues            | 0 (0 resolved) |
|           |                                  | Notes & Additional Information | 0 (0 resolved) |
|           |                                  | Client Reported Issues         | 0 (0 resolved) |

# Scope

OpenZeppelin performed an assessment of a regenerated v31 upgrade calldata package, published in the [l1-contracts/upgrade-envs/v0.31.0-interopB/output/mainnet](#) folder of [era-contracts](#) at commit [66ad9d57](#)

The subject of this review is the calldata itself rather than a source repository. Its authoritative source is not an on-chain governance proposal — at the time of this review the package had not yet been submitted to governance — but MatterLabs' own committed calldata-generation output, [ecosystem.toml](#), at [l1-contracts/upgrade-envs/v0.31.0-interopB/output/mainnet](#) of [era-contracts](#) at commit [66ad9d57](#).

The package carries three actions, each a [sendToL1\(bytes\)](#) call to the [L1Messenger](#) system contract on ZKsync Era, delivering one ABI-encoded [UpgradeProposal](#) to L1, one per governance stage. Together they comprise 42 calls:

| Stage | Calls | Encoded calls payload |
|-------|-------|-----------------------|
| 0     | 12    | 2,624 bytes           |
| 1     | 23    | 66,208 bytes          |
| 2     | 7     | 1,728 bytes           |

Every call, including nested structures such as diamond cuts, the L2 protocol upgrade transaction and the force-deployment lists, was decoded and evaluated against the contracts it targets.

The directory's own [REVIEW.md](#), dated 21 September 2026, identifies the era-contracts source this candidate was generated from as commit [bfdd283e](#) (generation run [35621948305](#)), and states it supersedes an earlier "optnone" candidate, with only four execution-data entries changed relative to it (the Era CTM upgrade registration, Era chain-creation parameters, and two Era chain smoke-test entries) — call ordering, targets, senders and values otherwise unchanged. OpenZeppelin used this same commit to verify the provenance of the deployed bytecode the calldata installs.

Per the same [REVIEW.md](#), this candidate's fork state is pinned to Ethereum block [25925815](#), predating the deployment broadcast — the block OpenZeppelin also used when

replaying the package end-to-end. As the package had not yet been submitted on-chain, live mainnet state used to cross-check individual findings (e.g. proxy initializer counters) was read at chain tip via RPC at the time of each check, rather than at a single fixed historical block.

This end-to-end replay, together with a run of MatterLabs' own Protocol Upgrade Verification Tool (PUVT) against the same package, completed with ten warnings — matching [REVIEW.md](#)'s own stated result exactly (nine chains on older protocol versions, and eleven historical deployments lacking explicit constructor-expectation coverage) — independently corroborating that the package OpenZeppelin reviewed is the one described by MatterLabs' own generation record.

# Calldata Verification Approach

This was an evaluation of the regenerated v31 upgrade calldata. We performed a fork verification while we also read the calldata call by call. All 42 calls across the three stages were decoded and checked against the contracts they target, the live state they depend on, and the source at the reviewed commit. For every call we inspected what it changes, whether its arguments are correct, whether its preconditions hold on every affected chain, and whether its position in the sequence matters.

In addition, we focused on confirming that the issues raised in OpenZeppelin's "v31 Upgrade Calldata Verification" report and the critical [ProtocolUpgradeHandler](#) reinitialization bug reported by the ML team separately have been addressed.

## Changes from the original calldata

- [oldProtocolVersion](#) updated. The Stage 1 call that registers the new protocol version now passes the chain's real, live protocol version. The previous calldata package included a stale value three versions behind it, which caused [execute\(\)](#) to revert with [OutdatedProtocolVersion](#).

- **Target protocol version updated.** The original package advanced both the Era and ZKsync OS chain type managers to the same version, v0.31.0. The regenerated package advances them to v0.32.2 and v0.31.2 respectively.
- **ProtocolUpgradeHandler implementation upgrade made atomic.** The Stage 1 call that installs the new ProtocolUpgradeHandler implementation now bundles the implementation swap and the `initialize(...)` call into a single `upgradeAndCall` transaction. The previous calldata package upgraded the implementation with empty initialization data.
- **Verifier address corrected.** The Stage 1 call that wires the chain's verifier now points to a newly deployed, correctly provenanced EraDualVerifier. The previous calldata package included a stale, pre-fix verifier address.
- **DA-scheme ABI inference fixed.** The off-chain tooling that infers whether a chain's diamond-cut data should be decoded against the ZKsync OS or Era DA commitment scheme no longer infers this from the chain's protocol version number; it now reads explicit, named minor-version constants.
- **Call order swapped for chain-creation-parameter registration.** `setNewVersionUpgrade` (which advances the CTM's live protocol version) is now called before `setChainCreationParams` (which stores its pointer under whatever protocol version is live at call time), for both the Era and ZKsync OS flavors. The previous calldata package called them in the reverse order.
- **Call count reduced from 45 to 42.** A net effect of the above changes to Stage 1's call sequence, not a change in scope of what the upgrade does.

# The critical issue:

## ProtocolUpgradeHandler reinitialization

The original package's Stage 1 call swapped ProtocolUpgradeHandler's implementation with empty initialization data. Because the live proxy's `_initialized` counter was already at 1 from genesis and the new implementation's `initialize(...)` is gated by `reinitializer(2)` (which requires `_initialized < 2`), the swap left that function callable by any account once the new implementation went live, with no accompanying initialization call to close the window. A successful call by an arbitrary actor would simultaneously seize the Security Council, Guardians, and Emergency Upgrade Board roles.

The regenerated package fixes this by issuing the same implementation swap through an atomic `upgradeAndCall`: the swap and the `initialize(...)` call (with the legitimate role addresses) execute in the same transaction. Since `reinitializer(N)` sets `_initialized` directly to `N` rather than incrementing it, `_initialized` moves from 1 straight to 2 within that single transaction, with no intervening state in which the function is reachable but uncalled.

We verified this directly rather than relying on source review alone: replaying the real Stage 0 sequence on a fork via impersonation to bring `ProtocolUpgradeHandler` to its true live state, then applying the regenerated package's Stage 1 call, and finally attempting the same unprivileged-attacker `initialize(attacker, attacker, attacker)` call that had succeeded against the previous calldata package. Against the regenerated package, that call correctly reverts.

# Status of previously reported issues

- **Stale pre-fix verifier.** Fixed. The regenerated package installs a newly deployed `EraDualVerifier`; its FFLONK and PLONK verification-key hashes were checked against the canonical genesis configuration and match exactly, and its deployment provenance was independently confirmed by PUVT's bytecode-recompilation check.
- **DA-scheme ABI inference from protocol version.** Fixed. The off-chain tooling now reads explicit minor-version constants for the Era and ZKsync OS DA commitment schemes instead of inferring the scheme from the chain's protocol version, which had caused incorrect decoding for at least one real mainnet chain under the original logic.
- **L2ChainAssetHandler force-deployment constructor-invocation flag.** Not fixed. The force-deployment list still requests constructor invocation for a contract that has no constructor. This was already assessed as low-risk: a dedicated `foundry-zksync` test confirmed that force-deployment converges to the same end state whether or not the constructor flag is set, so the anomaly is cosmetic rather than exploitable. It remains present, unchanged, in the regenerated package.

# Fork verification

We replayed the regenerated package end-to-end on a local Anvil fork of Ethereum mainnet pinned to block [25925815](#) — the same block the calldata-generation pipeline itself forked from, predating the real deployment broadcast. Using MatterLabs' own [protocol\\_ops](#) tooling, we funded and impersonated the real deploy-bundle signers, broadcast the full deployment bundle (the CREATE2 deployments of every new implementation contract, including the corrected verifier and [ProtocolUpgradeHandler](#) implementations), then executed the three governance stages through [ProtocolUpgradeHandler](#)'s real state machine, and finally ran the Protocol Upgrade Verification Tool (PUVT) against the resulting state.

The run completed with **ten warnings**, matching what we had been told to expect:

- **Nine warnings** for chains still running older protocol versions. These are pre-existing chains unrelated to this proposal that simply haven't been upgraded yet; PUVT flags any chain below the target protocol version, which is expected and not a defect in the calldata under review.
- **One warning** covering eleven indexed historical CREATE2 deployments that lack explicit constructor-expectation coverage in PUVT's own configuration. This is a gap in PUVT's own tooling coverage for older, already-deployed contracts, not a defect in the reviewed contracts or calldata.

No other warnings, errors, or failed checks were produced.

# Conclusion

No new issues were identified in the regenerated calldata package. The critical `ProtocolUpgradeHandler` reinitialization bug and two of the three previously reported issues are fixed and were independently verified, both by direct fork reproduction and by an end-to-end PUVT run; the third (the harmless constructor-invocation flag) remains present but was already assessed as low-risk.

Our verification confirms the calldata is internally consistent and correctly reproducible against a fork pinned to its intended starting state; it does not by itself confirm that the real CREATE2 deployment step will land at these same addresses against live state at the time the upgrade actually executes. That remains a separate check to perform once deployment occurs.

# Appendix

## Issue Classification

OpenZeppelin classifies smart contract vulnerabilities on a 5-level scale:

- Critical
- High
- Medium
- Low
- Note/Information

### Critical Severity

This classification is applied when the issue's impact is catastrophic, threatening extensive damage to the client's reputation and/or causing severe financial loss to the client or users. The likelihood of exploitation can be high, warranting a swift response. Critical issues typically involve significant risks such as the permanent loss or locking of a large volume of users' sensitive assets or the failure of core system functionalities without viable mitigations. These issues demand immediate attention due to their potential to compromise system integrity or user trust significantly.

### High Severity

These issues are characterized by the potential to substantially impact the client's reputation and/or result in considerable financial losses. The likelihood of exploitation is significant, warranting a swift response. Such issues might include temporary loss or locking of a significant number of users' sensitive assets or disruptions to critical system functionalities, albeit with potential, yet limited, mitigations available. The emphasis is on the significant but not always catastrophic effects on system operation or asset security, necessitating prompt and effective remediation.

### Medium Severity

Issues classified as being of medium severity can lead to a noticeable negative impact on the client's reputation and/or moderate financial losses. Such issues, if left unattended, have a moderate likelihood of being exploited or may cause unwanted side effects in the system.

These issues are typically confined to a smaller subset of users' sensitive assets or might involve deviations from the specified system design that, while not directly financial in nature, compromise system integrity or user experience. The focus here is on issues that pose a real but contained risk, warranting timely attention to prevent escalation.

### **Low Severity**

Low-severity issues are those that have a low impact on the client's operations and/or reputation. These issues may represent minor risks or inefficiencies to the client's specific business model. They are identified as areas for improvement that, while not urgent, could enhance the security and quality of the codebase if addressed.

### **Notes & Additional Information Severity**

This category is reserved for issues that, despite having a minimal impact, are still important to resolve. Addressing these issues contributes to the overall security posture and code quality improvement but does not require immediate action. It reflects a commitment to maintaining high standards and continuous improvement, even in areas that do not pose immediate risks.